Role-Based Access Control (RBAC), the model shifts: permissions are no longer granted to individual principals or teams for specific resources but to *roles*—that is, named collections of permissions that reflect job functions or organizational responsibilities. Principals are then assigned to one or more roles, gaining access indirectly through those assignments. This separation between identity and permission makes it easier to manage access at scale and keeps authorization logic aligned with the organization's structure.

Rather than maintain separate access rules for each user, an RBAC system defines a set of roles that reflect real-world job functions—such as `engineer`, `customer-support`, or `support-lead`—and assigns permissions to those roles. Users are then granted one or more roles based on their responsibilities. This approach simplifies change management: when a user changes roles, their access is updated automatically.

RBAC also supports role hierarchies, where one role can inherit the permissions of another. This is useful for modeling organizational seniority or layered responsibilities. For instance, the `senior-engineer` role might inherit permissions from the `engineer` role and add elevated privileges, such as deploying code to production. Role inheritance provides greater control of, and visibility into, access decisions by reducing the number of roles principals have.

5.2.1 How RBAC works

At runtime, an RBAC system determines whether a user is allowed to perform a particular action by evaluating their assigned roles and the permissions associated with those roles. The system doesn't check whether the user has direct access to the resource—it checks whether one of their roles grants the necessary permission.

When a user makes a request, the system typically performs the following steps:

1. Identify the user (principal) making the request.
2. Determine the roles assigned to that user.
3. Retrieve the permissions associated with those roles.